them on a request `https://api.example.com:4567/a` or other endpoints.

4.1.4 Drawbacks of HTTP authentication

Now that you've implemented a simple UI for the Natter API using HTTP Basic authentication, it should be apparent that it has several drawbacks from both a user experience and engineering point of views. Some of the drawbacks include the following:

- The user's password is sent on every API call, increasing the chance of it accidentally being exposed by a bug in one of those operations. If you are implementing a microservice architecture (covered in part 3), then every microservice needs to securely handle passwords.
- Verifying a password is an expensive operation, as you saw in chapter 3, and performing this validation on every API call adds a lot of overhead. Modern password-hashing algorithms are designed to take around 100ms for interactive logins, which limits your API to handling 10 operations per CPU core per second. You're going to need a lot of CPU cores if you need to scale up with this design!
- The dialog box presented by browsers for HTTP Basic authentication is pretty ugly, with not much scope for customization. The user experience leaves a lot to be desired.